KAVACH: An On-Device, Multilingual Scam-Call Shield for Indian Phone Users with Llama 3.2 1B on Arm CPUs

Siddhant Singh, Bogavalli Sai Akhilash, Shivansh Bose
Manipal Institute of Technology
Manipal Academy of Higher Education
Manipal, Karnataka, India

Dr. Rahul Ratnakumar
Faculty Mentor
Manipal Institute of Technology
Manipal Academy of Higher Education
Manipal, Karnataka, India

***Abstract*—Phone fraud in India increasingly happens on a *live call*: a caller poses as police, a bank or a courier, isolates the victim and steers a payment while the call is still running. Cloud-based detectors cannot hear such calls without sending private audio off the phone. This paper presents KAVACH, a single Android application that listens to a call the user has put on speaker, recognises Hindi, Hinglish and English speech on the phone, and warns the user *during* the call, with no internet permission at all. Speech is cut into sentences by a voice-activity detector and recognised by two int8 CTC models (AI4Bharat IndicConformer for Hindi, NVIDIA Parakeet 110M for English); an *Auto* mode runs both on every sentence and keeps the transcript whose words are real words of its language. A sub-5 ms detector (cue lexicon, hashed linear classifier and a multi-channel kill chain) raises CAUTION or STOP, and a LoRA-fine-tuned Llama 3.2 1B, quantised to 4 bits and run with ExecuTorch on the Arm CPU, explains the alert in the caller's language. On a mid-range Snapdragon 7s Gen 2 phone, both speech models together run about $5\times$ faster than real time, an hour of listening costs about 7.6 % battery, and Llama decodes 11.6–11.9 tokens/s. In three real cellular calls from an unknown number, a Hindi "digital arrest" scam and an English KYC scam reached STOP during the call and a genuine bank call raised no warning. Fine-tuning raised Llama's valid-verdict rate from 71 % to 100 % and scam accuracy from 38 % to 89 %.**

***Index Terms*—Scam detection, on-device AI, speech recognition, Llama, ExecuTorch, Arm, LoRA, Android, privacy, Hindi**

I. Introduction

Indian phone scams have shifted from single messages to orchestrated conversations. In a typical "digital arrest" call the fraudster claims to be from a cyber-crime branch, says a parcel booked on the victim's Aadhaar contains drugs, forbids the victim to tell anyone, and finally asks for a transfer to an "RBI safe account". Every step is spoken; nothing is written. A protection that only reads SMS or URLs arrives after the money has gone, and a protection that uploads call audio to a server creates a new privacy risk of its own.

KAVACH (Sanskrit for *armour*) takes the opposite route: everything runs on the user's phone. The design goals were (i) *on-device only*: the main app does not even request Android's `INTERNET` permission and never stores audio; (ii) *Indian languages*: Hindi, Hinglish and English, including callers who switch mid-call; (iii) *early and calm*: warn before the payment step, with zero false STOPs on genuine calls as a hard target; (iv) *explainable*: say in plain words, in the caller's language, what trick is being played and what to do (call 1930, the national cyber-crime helpline); (v) *Arm-efficient*: run in real time on a mid-range phone CPU with measured speed, memory, battery and heat.

Contributions.

- A one-phone architecture that listens to a speaker-phone call through Android's accessibility audio exception, with automatic start on calls from unknown numbers and a self-check that detects when Android silently stops the permission.
- A bilingual sentence-level speech front-end with per-sentence language choice (*Auto*), raising Hindi telephone-band detection from 0 % to 58 % STOP on 100 unseen calls.
- A fast detector (cues, hashed classifier, kill chain) with 0 % false STOPs on every evaluated set, and a LoRA-tuned Llama 3.2 1B advisor with a strict JSON contract and validator.
- Measurements on a real phone and three real cellular calls, with all code, data generators and results released.

II. Related Work

Large language models such as Llama 3 [1] can explain fraud patterns, but a phone needs a small, quantised model: SpinQuant [2] and groupwise 4-bit schemes make Llama 3.2 1B fit in under 1 GB, and ExecuTorch [3] with the XNNPACK backend and Arm KleidiAI micro-kernels [4] runs it on Arm CPUs. Low-rank adaptation (LoRA) [5] specialises such a model cheaply. For speech, Conformer [6] and Fast Conformer [7] encoders trained with CTC [8] dominate on-device recognition; AI4Bharat's IndicConformer, trained on large Indian-language corpora such as IndicVoices [9], covers Hindi, and NVIDIA's Parakeet [10] covers English; sherpa-onnx [11] provides int8 ONNX runtimes and the Silero voice-

This paper is the Phase 1 technical submission of team KAVACH to the **Arm Bharat AI-SoC Challenge 2026–27**, Problem Statement 3: *Cybersecurity and Digital Safety*. Source code, models, benchmarks and the full technical report: https://github.com/siddhaantxsingh/KAVACH.

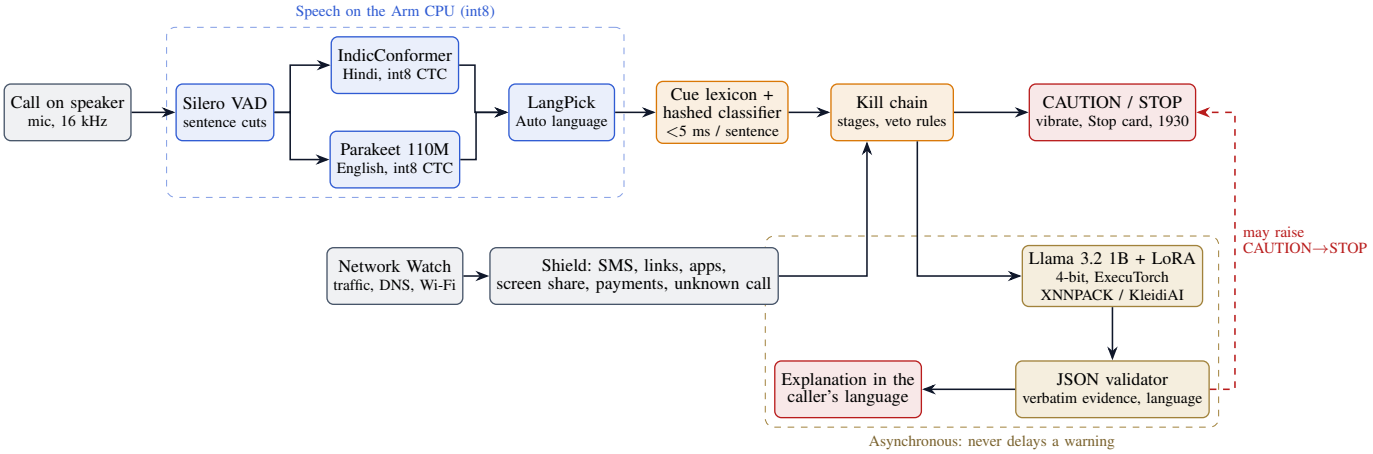


Fig. 1. KAVACH pipeline on one Android phone. Blue: speech front-end; amber: always-on detector; gold: Llama advisor; red: what the user sees. No component uses the network.

activity detector [12]. Our classifier uses feature hashing [13], and our multi-channel fusion borrows the *kill chain* idea from intrusion analysis [14]: a single cue is weak, a sequence of attack stages is strong. Unlike server-side call screening, KAVACH keeps audio, transcripts and model outputs on the phone.

III. System Architecture

Fig. 1 shows the pipeline. Three on-device sources feed one kill chain: *call listening* (what is said), *Shield* (messages, links, apps, screen sharing, payment apps, and the fact that an unknown number is calling) and the optional *Network Watch* app (per-app traffic, website look-ups and risky devices on the home Wi-Fi).

A. One-phone call listening

During a cellular call Android gives the microphone to the call and feeds every other app's recording with silence. The documented exception is an enabled accessibility service [15]. KAVACH declares a minimal service that reads no screen content and performs no actions; while it runs, the app keeps hearing a call the user has put on speaker. A phone-state listener and a contacts look-up start listening automatically when an *unknown* number calls. During our tests Android stopped the app once (after the default dialer changed) and marked the service as crashed while its switch still showed "on"; every call recording was then silenced. The app now compares the enabled setting with the services Android actually runs, and checks `isClientSilenced()` on its own recording during a call, telling the user how to fix it instead of failing silently.

B. Speech front-end and Auto language

Silero VAD cuts audio into sentences (0.5 s minimum silence, 12 s maximum). Each sentence is recognised whole by an int8 CTC model, with a partial result every 2 s while the caller speaks. In *Auto* mode both models recognise every sentence, and LangPick keeps the transcript whose words are more often real, common words of its language:

$$s = (k_{en} - k_{hi}) + W\,(c_{en} - c_{hi}), \tag{1}$$

where k is the share of words among the 20,000 most frequent words of that language, c the share among the 200 most frequent (mostly function words), both counted over $n{+}1$ words so that a one-word fragment cannot win, and $W{=}1$. English is chosen if $s > 0.2$; a sentence within 0.15 of the margin keeps the previous sentence's language, and warnings follow the majority of the last three sentences. Hinglish naturally goes to the Hindi model, which writes English loanwords that the lexicon also knows in Devanagari.

C. Always-on detector

A cue lexicon (with 208 mixed-script loanword variants mined from what the Hindi model actually writes) marks authority claims, identity hooks, threats, isolation, credential and payment requests. A hashed linear classifier [13] scores the last 120 tokens. The kill chain combines cues over time and across channels into CAUTION or STOP; context rules veto false alarms, for example "never tell anyone your OTP" is safety advice, not isolation. One update costs about 4 ms on the phone.

D. Llama advisor

On CAUTION or STOP, the transcript window, active cues and the three most relevant playbook entries are given to Llama 3.2 1B [1]. The constant system prefix (rules, schema, scam types) is pre-filled into the KV cache while the call starts. The model must return one JSON object (`scam`, `type`, `stage`, `confidence`, `evidence`, `say`, `action`); a validator keeps only evidence quoted verbatim from the transcript, rejects advice that repeats the caller, replaces advice that copies the prompt's example or is in the wrong language, and drops an unfinished last sentence. Llama can raise CAUTION to STOP but can never lower a rule STOP.

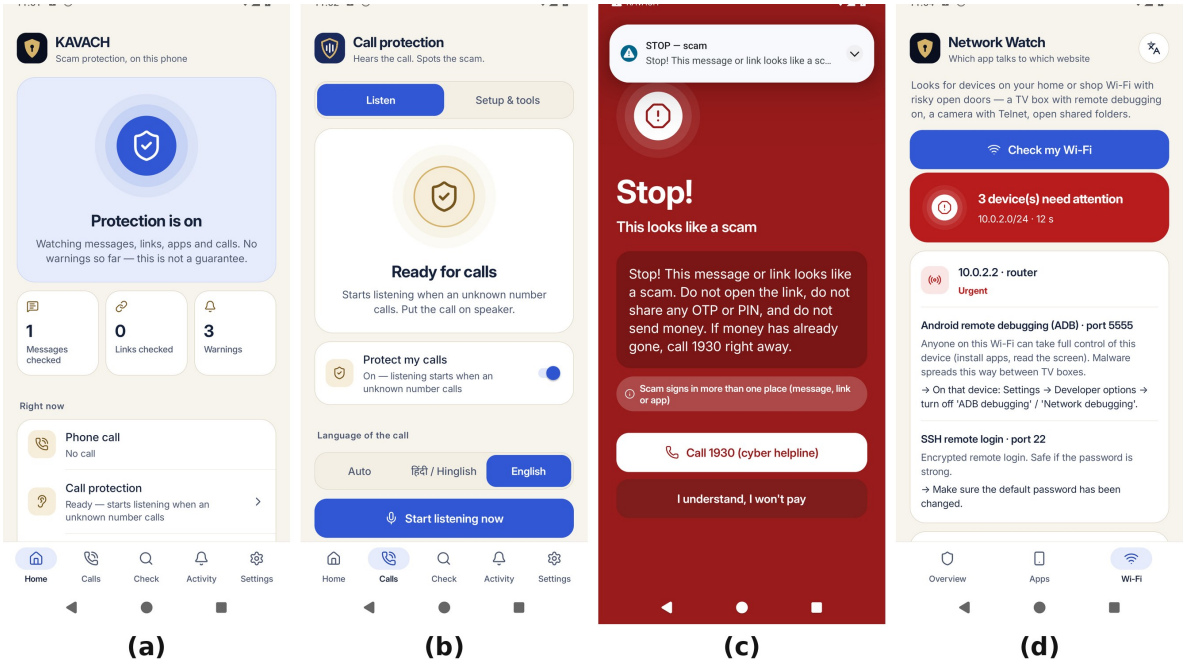


Fig. 2. The app. (a) Home with live protection status and on-phone counters. (b) Call protection, Auto language. (c) Stop card raised by a scam SMS and link. (d) Network Watch finding risky devices on the home Wi-Fi.

Fine-tuning. We generated 4,000 training and 400 development instruction pairs with the same prompt code the phone runs, trained LoRA [5] adapters (r=16, all projections, 1 epoch, free Colab T4 GPU, 4.1 h), merged them, and exported the model with ExecuTorch 1.4 using 8-bit dynamic activations and 4-bit groupwise weights (g=32) plus 4-bit embeddings, giving an 820 MB file (Meta's SpinQuant file is 1,083 MB).

E. Privacy and safety

The main app has no `INTERNET` permission, audio lives only in memory, and Network Watch, which needs network access for its DNS watch, is a separate optional app. KAVACH never blocks payments; it warns, explains and offers the 1930 helpline.

IV. Evaluation

A. Setup

Host experiments use synthetic calls generated from scam playbooks (8,000 train / 600 dev / 1,000 test calls; a 20-call locked held-out set touched only for reporting), spoken by Piper TTS [16] through a simulated telephone band and room noise. Phone experiments use a Xiaomi 2312DRA50I (Snapdragon 7s Gen 2: 4×Cortex-A78 at 2.4 GHz + 4×Cortex-A55 at 1.96 GHz, 7.3 GB RAM, Android 15), on battery.

B. Call-level detection (host)

Table I replays whole calls in call time. The shipped detector meets the targets of a median time-to-STOP under 30 s and at most 2 % false STOPs on every set.

TABLE I
CALL-LEVEL RESULTS OF THE SHIPPED DETECTOR

Set (scam / genuine)	STOP	≥CAUT.	Median to STOP	False STOP
Synthetic test (511 / 489)	90.6 %	97.3 %	12.7 s	**0 %**
Through speaker (502 / 498)	71.1 %	88.8 %	14.7 s	**0 %**
Hand-written (16 / 14)	93.8 %	93.8 %	17.0 s	**0 %**
Locked held-out (10 / 10)	70.0 %	80.0 %	22.6 s	**0 %**

C. Hindi speech

Hindi was initially the weak link: the small streaming model turned telephone-band speech into mostly unusable text. Fig. 4 compares it with IndicConformer on 100 unseen Hindi calls (50 scam, 50 genuine). With IndicConformer, genuine calls reached STOP 0 % of the time and CAUTION 6 % (a traffic-police call and two genuine card-fraud checks).

D. Auto language

On the held-out split (first two caller sentences of each call, both channels), LangPick chose the right model for 92.7 % of sentences (345/372): Hindi 91.0 % / 99.0 % and English 93.0 % / 87.2 % on the telephone / near-speaker channels. Most errors are short English sentences whose Hindi transcript is a one- or two-word fragment. On the phone, all four English test calls and both Hindi test calls replayed from a laptop speaker behaved as expected (scams STOP, genuine calls calm).

E. Llama before and after fine-tuning

Fig. 5 scores the base and the fine-tuned model on 100 development examples with `eval_llama.py`. The two failures seen on the phone before fine-tuning, advice copied from

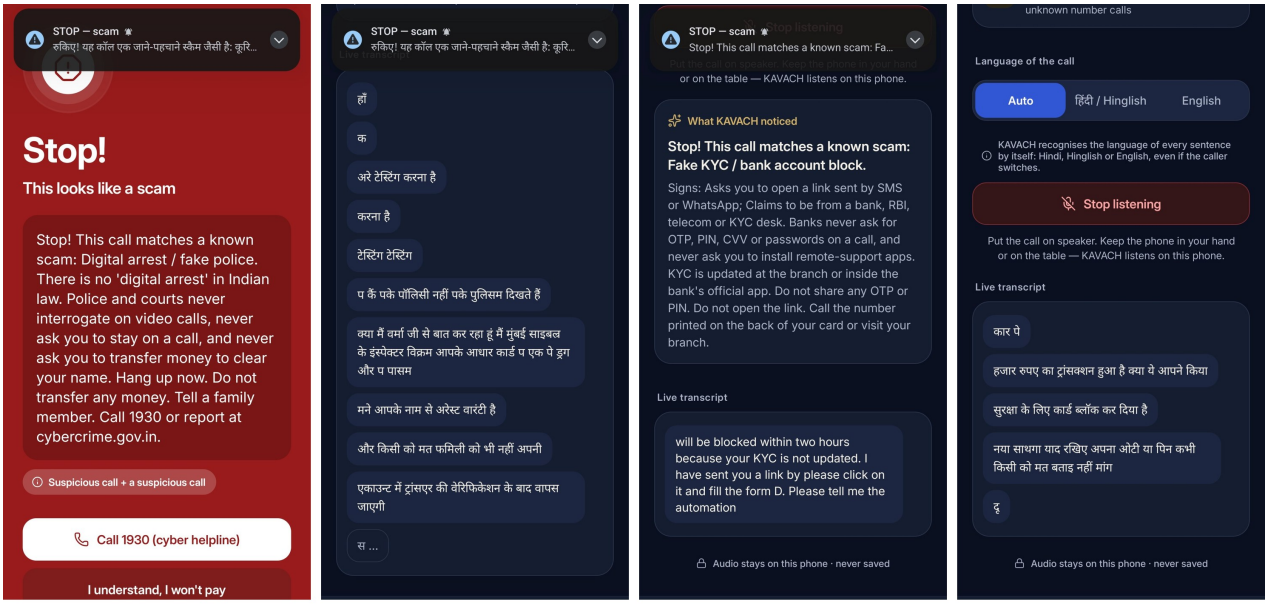


Fig. 3. Real cellular calls from an unknown number, one phone, call on speaker, language on Auto. (a) Hindi digital-arrest scam: full-screen Stop card during the call. (b) Its live transcript through the phone line. (c) English KYC scam: STOP, "Fake KYC / bank account block". (d) Genuine Hindi bank fraud-check call: no warning.

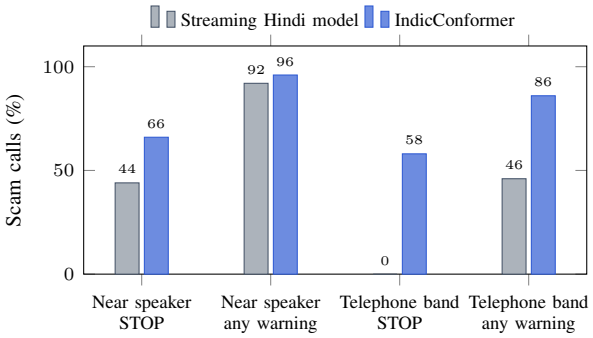


Fig. 4. Hindi scam calls detected, 100 unseen calls per condition.

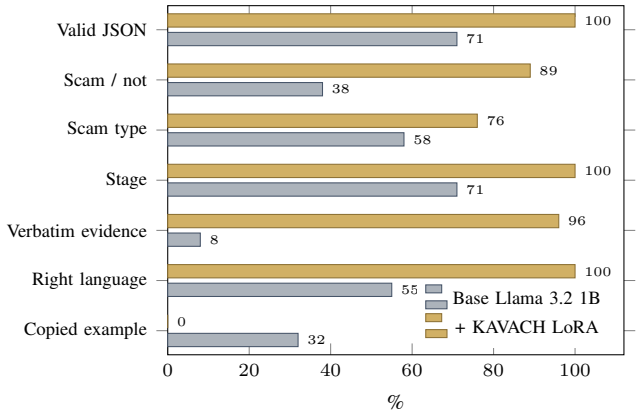


Fig. 5. Llama verdict quality before and after LoRA (higher is better, except "copied example").

the prompt's example and English advice during a Hindi call, fell from 32 % to 0 % and rose from 55 % to 100 % correct-language respectively. These examples share the generator with the training data, so they measure learning of the task, not generalisation to human speech.

F. Performance on the phone

Table II lists the on-phone benchmark (`BenchmarkActivity`, on battery). Real-time factor (RTF) is processing time divided by audio time. In Auto mode the two sentence models together use about $0.15{+}0.05{\approx}0.2$ of real time at two threads each. Four threads make the big models clearly faster (Hindi RTF $0.153{\rightarrow}0.092$); the app uses two to save battery.

G. Real calls on one phone

A teammate called the test phone from a number not in its contacts over the cellular network; the call was answered in the phone's own dialer and put on speaker, and KAVACH started by itself with the language on Auto. Fig. 3 shows the results: the Hindi digital-arrest scam reached STOP during the call with a transcript close to the script; the English KYC scam reached STOP with the correct type; a genuine Hindi bank fraud-check call (which mentions the card, a transaction, OTP and PIN) raised no warning. Fig. 6 shows the fine-tuned model on the phone writing its own advice in English and in Hindi.

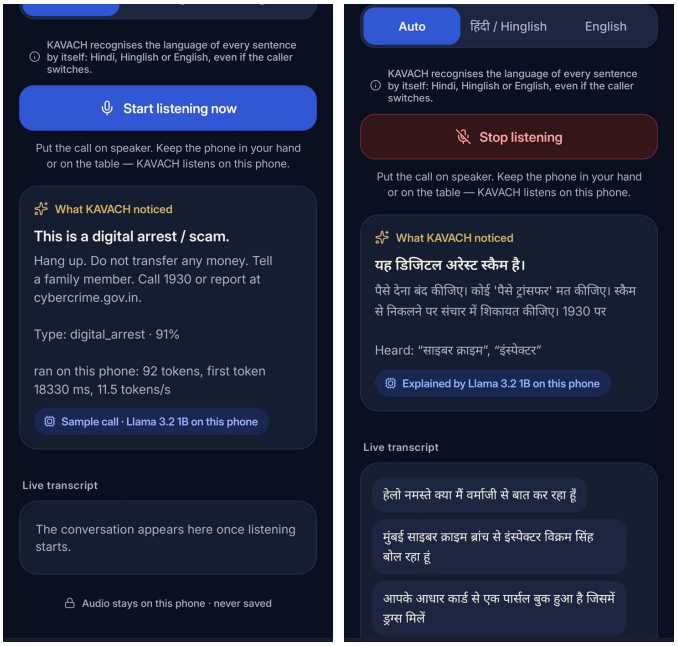


Fig. 6. Fine-tuned Llama 3.2 1B on the phone. (a) Self-test: type *digital_arrest*, 0.91, own advice, 11.5 tokens/s. (b) During a Hindi call: own Hindi advice with verbatim evidence, "Explained by Llama 3.2 1B on this phone".

TABLE II
MEASURED ON THE PHONE (SNAPDRAGON 7S GEN 2, ON BATTERY)

Component	Result
Detector	4.07 ms per sentence
IndicConformer Hindi	RTF 0.153 (2 thr), 0.092 (4 thr)
Parakeet 110M English	RTF 0.052 (2 thr), 0.037 (4 thr)
Auto listening, 1 min	54 % of one core, 385 mA
	≈7.6 % battery/hour, 38.0→37.9 °C
Llama load	4.1 s, +1.1 GB
Llama first token	5.9 s warm prefix, 12.9 s cold
Llama prefill / decode	56–60 / 11.6–11.9 tokens/s
Llama while generating	858 mA, no thermal throttling

V. LIMITATIONS AND ETHICS

All accuracy numbers except the three real calls use synthetic text or TTS voices from one voice per language; human voices, accents and crowded rooms will score lower, and three calls are a demonstration, not a statistic. The always-on detector misses about 10–30 % of scam calls depending on the set, and the "warned before payment" rate (50–78 %) is below our 90 % target. Llama's Hindi is understandable but not always polished. Listening requires the user to tap Speaker and to keep the accessibility permission running. Role-play recordings are collected only with written consent (`demo/consent_form.md`), and no real victim data is used.

VI. CONCLUSION AND FUTURE WORK

KAVACH shows that a mid-range Arm phone can hear, understand and explain a scam call in Hindi or English while the call is happening, entirely offline: two speech models, a millisecond detector and a 1B-parameter LLM together use about half of one CPU core and under 8 % battery per hour. Phase 2 will add a consented human-recorded test set, NPU offload of the speech encoders with CPU-vs-NPU energy measurements, per-rung quantisation and KleidiAI ablations, and on-device personalisation of the playbook.

ACKNOWLEDGMENT

The authors thank Dr. Rahul Ratnakumar for his guidance, the organisers of the Arm Bharat AI-SoC Challenge 2026–27, and the open-source projects this work builds on: Meta Llama, ExecuTorch, AI4Bharat IndicConformer, NVIDIA Parakeet, sherpa-onnx, Silero VAD, Piper and wordfreq.

REFERENCES

[1] A. Grattafiori *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
[2] Z. Liu *et al.*, "SpinQuant: LLM quantization with learned rotations," *arXiv preprint arXiv:2405.16406*, 2024.
[3] PyTorch Team, "ExecuTorch: On-device AI inference for PyTorch," 2025. [Online]. Available: https://github.com/pytorch/executorch
[4] Arm Ltd., "KleidiAI: micro-kernels for AI workloads on Arm CPUs," 2025. [Online]. Available: https://gitlab.arm.com/kleidi/kleidiai
[5] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.
[6] A. Gulati *et al.*, "Conformer: Convolution-augmented transformer for speech recognition," in *Proc. Interspeech*, 2020, pp. 5036–5040.
[7] D. Rekesh *et al.*, "Fast Conformer with linearly scalable attention for efficient speech recognition," in *Proc. IEEE Autom. Speech Recognit. Underst. Workshop (ASRU)*, 2023.
[8] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2006, pp. 369–376.
[9] T. Javed *et al.*, "IndicVoices: Towards building an inclusive multilingual speech dataset for Indian languages," in *Findings Assoc. Comput. Linguist. (ACL)*, 2024.
[10] NVIDIA, "parakeet-tdt_ctc-110m," model card, 2025. [Online]. Available: https://huggingface.co/nvidia/parakeet-tdt_ctc-110m
[11] k2-fsa, "sherpa-onnx: speech recognition, synthesis and VAD with ONNX Runtime," 2025. [Online]. Available: https://github.com/k2-fsa/sherpa-onnx
[12] Silero Team, "Silero VAD: pre-trained enterprise-grade voice activity detector," 2024. [Online]. Available: https://github.com/snakers4/silero-vad
[13] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg, "Feature hashing for large scale multitask learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2009, pp. 1113–1120.
[14] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains," Lockheed Martin Corp., Tech. Rep., 2011.
[15] Android Developers, "Share audio input," 2025. [Online]. Available: https://developer.android.com/media/platform/sharing-audio-input
[16] Rhasspy, "Piper: a fast, local neural text to speech system," 2024. [Online]. Available: https://github.com/rhasspy/piper